

2.7 Checkpoint: demo your backend

Explanation of project

The project is a **serverless cold-chain monitoring system** built using AWS services. Its purpose is to accept temperature readings for individual branches, validate and store those readings, determine whether they are within the permitted temperature range, and automatically trigger an alert workflow when an excursion occurs. The safe temperature range used by the application is **2°C to 8°C**. If `minTempC` falls below `2` or `maxTempC` rises above `8`, the reading is classified as an `"excursion"`; otherwise, it is classified as `"ok"`.

The system is tested using **Postman**, which acts as an API client. Postman sends HTTP requests to the API, allowing us to test the application without building a frontend. For example, a `POST` request is used to create a new temperature reading, while `GET` requests retrieve readings that have already been stored. Postman also allows us to test validation and error handling by deliberately sending invalid data and checking that the correct HTTP status codes are returned.

The public entry point to the system is **Amazon API Gateway**. We created an HTTP API containing routes such as:

```
1 POST /branches/{branchId}/readings
2 GET /branches/{branchId}/readings
3 GET /branches/{branchId}/readings/{recordedAt}
```

API Gateway receives the HTTP request, identifies the matching route and forwards the event to the Lambda function. The `{branchId}` and `{recordedAt}` sections are **path parameters**, which allow the API to dynamically access different branches and individual readings.

The main backend logic runs inside the `umar-coldchain-LinkDynamoDB` **AWS Lambda function**. Lambda provides serverless compute, meaning AWS runs the code only when a request invokes the function rather than requiring us to maintain a continuously running server. The Lambda validates incoming request data, checks required fields, validates timestamps and temperature values, detects duplicate readings, generates a unique `readingId`, and calculates the reading status using the cold-chain business rule.

For a successful POST request, the Lambda generates a composite sort key called `readingKey`. This contains the timestamp and the generated reading ID, for example:

```
1 2026-08-19T15:20:00.000Z#37a0e90b-f7d5-45d0-b6b8-dd6bd8e94216
```

The reading is then stored in **Amazon DynamoDB**. DynamoDB is the project's NoSQL database. The table uses `branchId` as the **partition key** and `readingKey` as the **sort key**. This design groups readings belonging to the same branch while allowing the readings to be ordered by timestamp.

A typical stored item contains data such as:

```
1 {  
2   "branchId": "BRANCH-001",  
3   "readingKey": "2026-08-19T15:20:00.000Z#...",  
4   "readingId": "...",  
5   "recordedAt": "2026-08-19T15:20:00.000Z",  
6   "minTempC": 3,  
7   "maxTempC": 12,  
8   "recordedBy": "Umar",  
9   "status": "excursion"  
10 }
```

The Lambda also handles application validation and returns appropriate **HTTP status codes**. A valid new reading returns `201 Created`. Invalid input returns `400 Bad Request`, duplicate readings return `409 Conflict`, a reading that cannot be found returns `404 Not Found`, and an unexpected server-side failure returns `500 Internal Server Error`.

The alert workflow begins only when:

```
1 | status === "excursion"
```

After the DynamoDB item is successfully stored, the create-reading Lambda publishes an event to an **Amazon SNS topic** called:

```
1 | umar-coldchain-excursions
```

SNS, or **Simple Notification Service**, uses a publish/subscribe model. The Lambda acts as the publisher and sends an excursion message containing information such as the branch ID, reading ID, timestamp and temperature values. SNS then distributes that message to its subscribers.

The SNS topic has an **Amazon SQS queue** subscribed to it:

```
1 | umar-coldchain-alert-queue
```

SQS acts as a message buffer between the publishing Lambda and the alert-processing Lambda. Instead of requiring the first Lambda to directly call the alert Lambda, SNS places the message into SQS. This provides **decoupling** between the components. If the downstream alert processing temporarily fails, the message remains available in the queue and can be retried rather than being immediately lost.

The main SQS queue is configured with a **dead-letter queue**, or DLQ:

```
1 | umar-coldchain-alert-dlq
```

If a message repeatedly fails processing, SQS eventually moves it into the DLQ. This prevents permanently failing messages from being retried forever and provides somewhere to inspect failed events. During testing, we deliberately caused the alert Lambda to throw an error and confirmed that the failed message eventually appeared in this DLQ.

The SQS queue is connected to a second Lambda called:

```
1 | umar-coldchain-excursion-handler
```

This Lambda is the **consumer** of the queue. AWS Lambda uses an SQS event source mapping to poll the queue and automatically invoke the handler when messages are available. Therefore, the alert-processing workflow does not require any manual action.

When the excursion handler receives the message, it uses the `branchId` and `readingKey` contained in the message to locate the original DynamoDB item. It then performs a `DynamoDB UpdateItem` operation and adds:

```
1 | alertRaisedAt
```

with the current timestamp.

For example, the final record might contain:

```
1 | {  
2 |   "status": "excursion",  
3 |   "alertRaisedAt": "2026-08-19T15:30:16.157Z"  
4 | }
```

The presence of `alertRaisedAt` provides an **audit record** showing that the system processed the excursion and raised an alert.

The handler also implements **idempotency**. SQS provides **at-least-once delivery**, which means the same message can occasionally be delivered more than once. Without protection, the system could therefore process one fridge excursion multiple times.

To prevent this, the DynamoDB update uses a condition equivalent to:

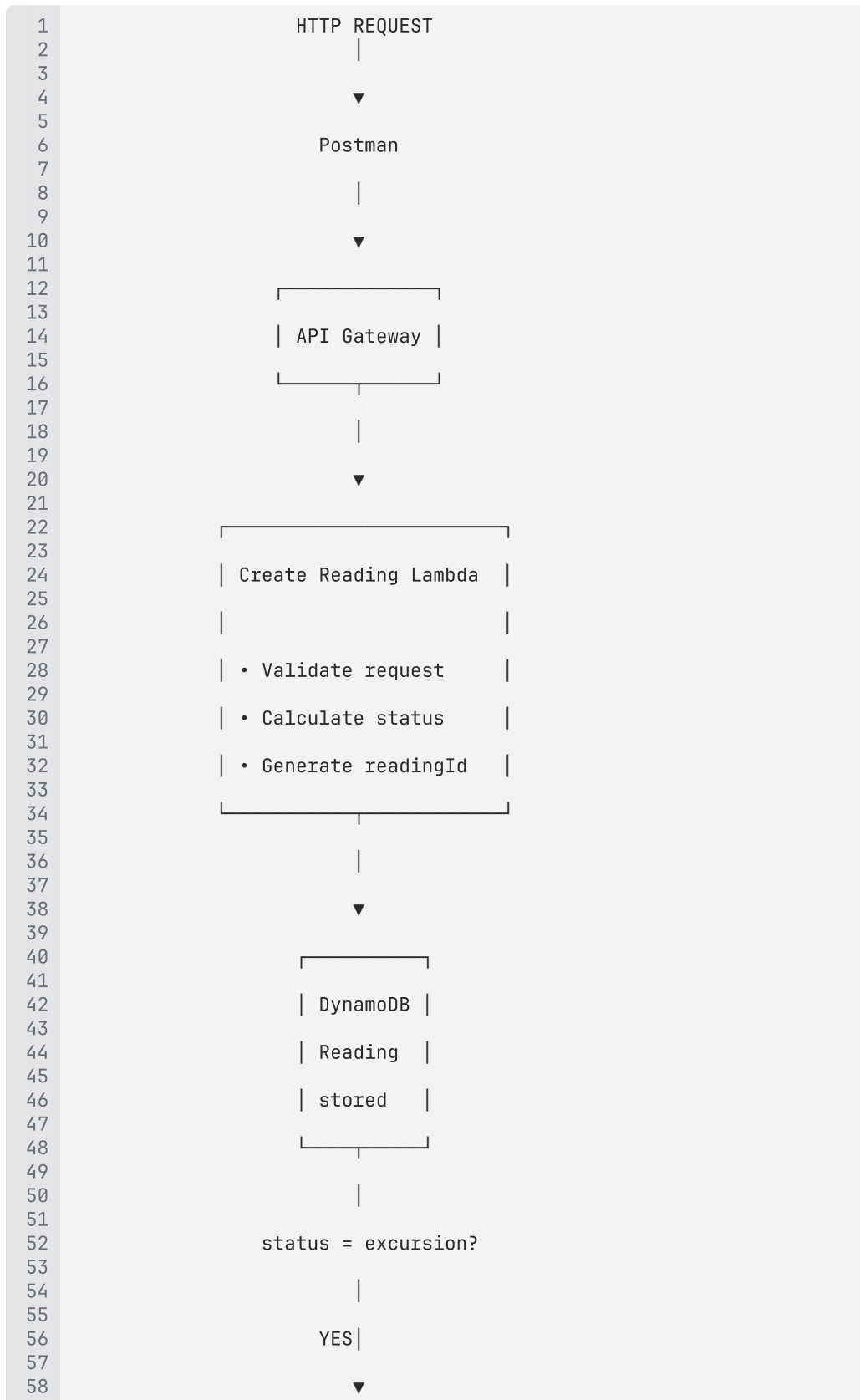
```
1 | ConditionExpression:  
2 | "attribute_not_exists(alertRaisedAt)"
```

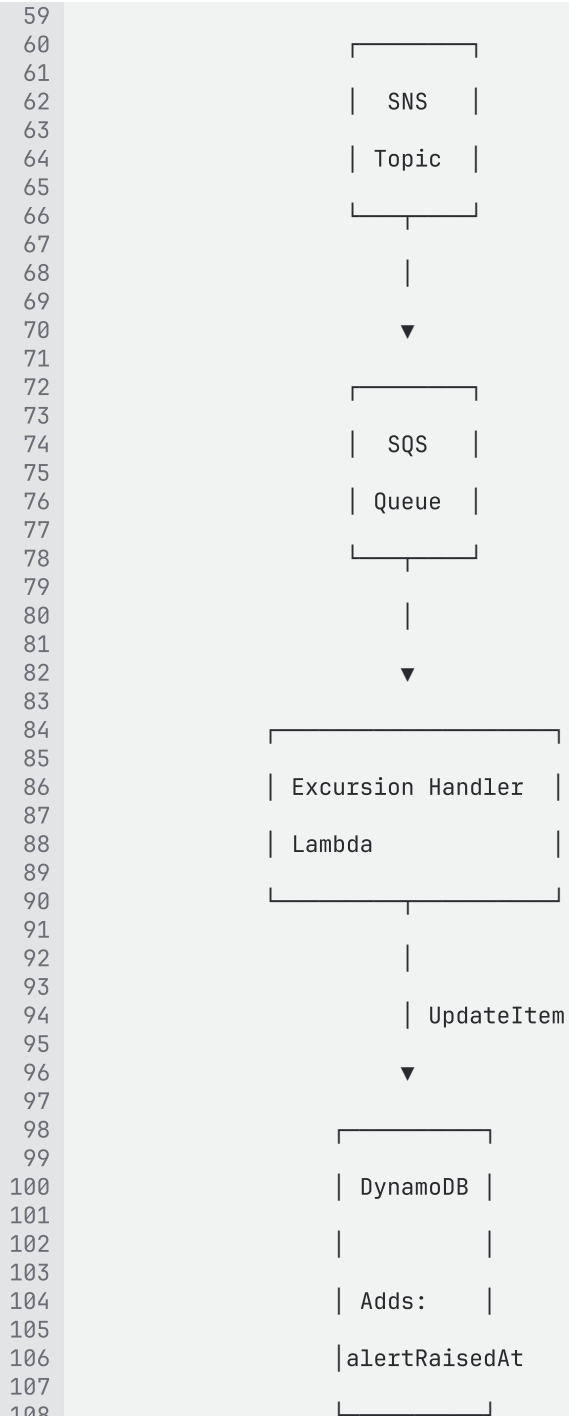
This means DynamoDB will only set `alertRaisedAt` if that attribute does not already exist. If the same SQS message is processed again, DynamoDB rejects the second update and the Lambda treats it as a duplicate. As a result, the original alert timestamp remains unchanged and only one logical alert is recorded.

The system also uses **Amazon CloudWatch** for observability. Both Lambda functions write logs into CloudWatch. This allows us to see when requests were received, when readings were

created, when excursion messages were processed, and when errors occurred. CloudWatch was particularly useful during development because errors such as IAM permission failures could be identified from the Lambda logs.

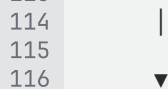
The overall architecture is:





110 If processing repeatedly fails:

111 SQS Main Queue



118 Excursion Handler



```
126     ▼
127
128     Retry
129
130     |
131
132     ▼
133
134 Dead-Letter Queue
```

So technically, the project demonstrates a **serverless, event-driven architecture** using synchronous HTTP communication for the API layer and asynchronous messaging for the alert workflow. API Gateway and Lambda handle the REST API, DynamoDB provides persistent NoSQL storage, SNS publishes excursion events, SQS provides durable asynchronous message processing, the DLQ handles repeated failures, CloudWatch provides logging and monitoring, and IAM controls which AWS resources each Lambda is authorised to access.

Demo

Showing the structure

```
1      Postman
2      ↓
3      API Gateway
4      ↓
5      Create Reading Lambda
6      ↓
7      DynamoDB
8      |
9      └─ if status = "excursion"
10
11          ↓
12          SNS
13          ↓
14          SQS
15          ↓
16          Excursion Handler Lambda
17          ↓
18          DynamoDB
19          ↓
20          alertRaisedAt
21
22 If processing keeps failing:
23 SQS
24 ↓
25 Excursion Handler Lambda
26 ↓
27 failure/retries
28 ↓
29 Dead-Letter Queue
```

Postman represents a client sending temperature readings. API Gateway is the entry point to the backend. The create-reading Lambda validates the request, calculates whether the temperature is okay or an excursion, and stores it in DynamoDB. If it is an excursion, the Lambda publishes a message to SNS. SNS forwards that message to SQS. The second Lambda consumes the SQS

message and updates the original DynamoDB record with `alertRaisedAt` . If processing repeatedly fails, the message eventually goes into the dead-letter queue.

Show the three API endpoints (R1, R2, R3)

Below are the requirements that i was working towards. (from Onboarding Project — Cold Chain Log)

#	Requirement
R1	A member of staff can record a temperature reading for a fridge at a branch
R2	Anyone can see the list of readings for a branch, newest first, and can filter to a date range
R3	Anyone can open a single reading and see everything about it
R4 (SNS deals with this)	When a reading is outside 2–8°C, the system decides that on its own and raises an alert without anybody asking it to

(R1) POST Create reading = This creates a new reading. Because 3°C to 7°C is inside the permitted 2°C to 8°C range, the server calculates the status as `ok` (showcase the DynamoDB new readings, show the 201 create and say 'this shows its successful').

(R2) GET Whole branch reading = This retrieves the readings for the Branch-001. (DynamoDB returns them newest first).

(R3) GET One specific reading = This retrives a specifide reading from a Branch-001.

Validation (not an endpoint just showing that validatino works)

POST Failure case = For the failure case, I use the same POST endpoint but sent invalid data where the maximum temperature is lower than the minimum. (The API rejects it 400 Bad

Request, which proves the backend validation is working, showing random data can not be passed through)

SNS and SQS showcases (R4)

Now showcase the SNS function:

POST SNS alert = request aiming to show when temps are above or below the min and max an excursions will send and will cause an alert to be raised in the DynamoDB. Aiming to show the structure below:

Plain Text

```
1 Create Lambda
2   ↓
3 SNS: "an excursion happened"
4   ↓
5 SQS holds the message
6   ↓
7 Excursion Handler Lambda processes it
8   ↓
9 DynamoDB gets alertRaisedAt
```

(open the dynamoDB and shows the alert column. also show how the "POST create reading = value ok" doesn't have the alert but this excursion one does)

Now showcase the SQS function:

Open: umar-coldchain-alert-queue

Say: "The SQS is in between the SNS messaging process and the lambda processing. I have done this to sort of separate the two parts so that if the second lambda fails then, the work isn't immediately lost because the message can be held by this SQS"

--

Then Open: umar-coldchain-alert-dlq

I already have my deliberate failed message in there so talk about that one. Say:

"I deliberately sent a message with `forceFailure: true`. The excursion Lambda threw an error, SQS retried the message, and after the maximum receive count was exceeded (maximum receive count was set to 3) it moved into this dead-letter queue. That gives us somewhere to investigate messages that cannot be processed."

Safe to run twice

DEF: Idempotency is the property of certain operations or functions where applying them multiple times gives the same final result as applying them just once.

--

I can't remember if this was mentioned on a ticket or I came up with a fix for this (must have been mentioned in one of the tickets.). But essentially this was solving if someone was to send the same message twice. I tested this by sending the same excursion message twice. The handler uses a DynamoDB condition that only adds `alertRaisedAt` if the attribute does not already exist. Therefore the second delivery does not overwrite the original alert or create a second one. That makes the handler idempotent.

the code that does it is got too: Lambda Code and look for :

```
UpdateExpression: "SET alertRaisedAt = :alertRaisedAt", ConditionExpression:
"attribute_not_exists(alertRaisedAt)", ExpressionAttributeValues: { ":alertRaisedAt": alertRaisedAt
```

Say: The code creates the alert and attaches it to that specific reading. If the same message arrives again, DynamoDB sees that `alertRaisedAt` already exists and refuses the second update

What I found difficult and what I improved

I am also going to be completely honest and say that a lot of the more advanced JavaScript code in this project was written with the help of AI. In particular, things such as the asynchronous AWS SDK code, the SNS and SQS handling, and especially the DynamoDB conditional update used for idempotency were beyond what I would currently be able to write completely by myself. I am still learning how to write JavaScript properly, so I do not want to pretend that I wrote every line without help. However, I have tried to understand what the code is doing rather than just copying it. This project has massively increased my understanding of coding and AWS compared with where I was at the beginning. For example, when I first started the project I did not even properly understand basic things such as what a Lambda `handler` was, whereas now I can explain what the handler does, what the `event` contains, how the Lambda communicates with other AWS services and how the different parts of the system connect together.

Another area I found difficult was understanding the asynchronous messaging flow between SNS, SQS and the excursion-handler Lambda. I had to understand that SNS is responsible for publishing the excursion event, SQS stores the message until it can be processed, and then the second Lambda consumes that message and updates the DynamoDB record. This was different

from the API Gateway flow because the alert process does not happen as one direct request from start to finish. Instead, each part of the system completes its own job and passes the work on to the next service.

CloudWatch also became very important while I was troubleshooting the project. At one point, the excursion-handler Lambda was receiving the SQS message correctly, but CloudWatch showed an `AccessDeniedException`. This helped me understand that the SNS and SQS parts were working and that the problem was specifically with the Lambda not having permission to update DynamoDB. After adding the required `dynamodb:UpdateItem` permission to the Lambda execution role, the alert flow worked and `alertRaisedAt` appeared automatically on the DynamoDB record.

I also found the idea of retries, visibility timeouts and dead-letter queues slightly difficult at first. I learned that when a Lambda fails to process an SQS message, the message is not immediately lost. It becomes available again after the visibility timeout and can be retried. If it continues to fail after the configured number of receives, it is moved into the dead-letter queue so that the failed message can be investigated instead of being retried forever.

Another concept I had to spend time understanding was idempotency. Because SQS uses at-least-once delivery, the same message can sometimes be delivered more than once. This means the Lambda has to be safe to run twice. I solved this by using a DynamoDB `ConditionExpression` that only sets `alertRaisedAt` if that field does not already exist. This means the first message raises the alert, but if the same message is delivered again, DynamoDB prevents the timestamp from being overwritten and the system does not create a duplicate alert.

Unresolved issue / area to revisit

The one mistake that was made that I didn't fix was that API requests could be sent from Postman without the need for the API key from the gateway. The system itself is working correctly, but the API is currently publicly callable by anyone who knows the invoke URL. I understand this is a major security and authenticity risk. The functionality is complete and working, but authentication is one area I would revisit. At the moment the HTTP API is publicly callable if someone knows the URL. I investigated API keys, but adding AWS's native API-key functionality would mean changing the API Gateway type, so I chose not to risk breaking the working project just before the demo.

 How the demo went = (okay)

The demo went okay i believe i could have prepared more. and if i write the paragraphs for any presentation or demo i will be able to explain it alt better.

⚠ Feedback

Why it is taking so long to load (optimises the code).

Why have I used DQL.

What is fifo message. (first come first sever for messages)

Research = GSLI scan vs quires.

Add API keys to making it more secure (change te HTTPS to RESTful APIs and chek setting to see i you can change it to private).

✖ Main error / Improvements

The main point that I took away was DON'T JUST LET AI DO EVERYTHING FOR YOU (they could tell): rai said, you always have to be reading the document/code AI gives you as it is not effecting. If you let it do what it wants it will make scripts with 100s of lines (which it did) and it does it in the most inefficient way possible.

YOU always have to be inquisitive and constantly improving the code and looking over it.

OVERALL

Have a lot to improve on for my next demo. They will keep questioning me and that is a good thing; next time don't just go erm I don't know, just say I am unsure I will look into it or something.

Feedback answers

Why it is taking so long to load (optimises the code). =

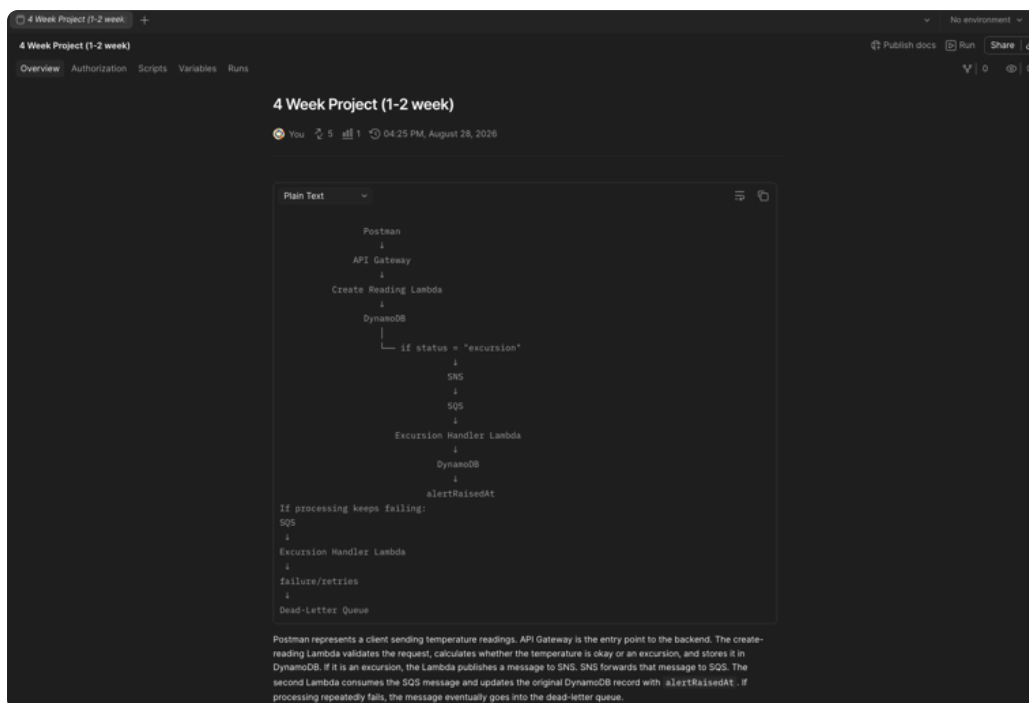
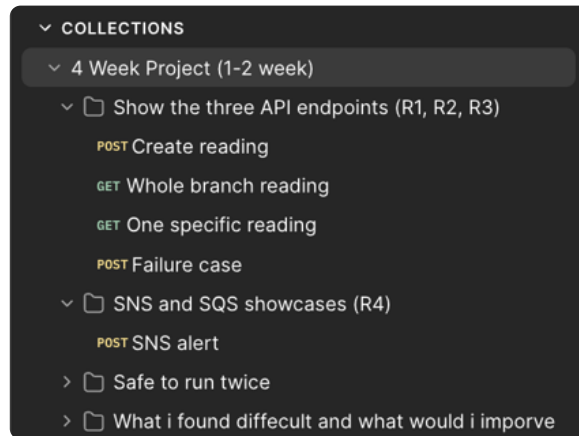
What is fifo message. = is a queue-based communication method where messages are processed strictly in the **First-In, First-Out** order. It makes sure if you submit a large message first then a small message either does it in the order yo uploaded them. so no matter what the larger file will be uploaded first . (Does it in order, first come first sever for messages)

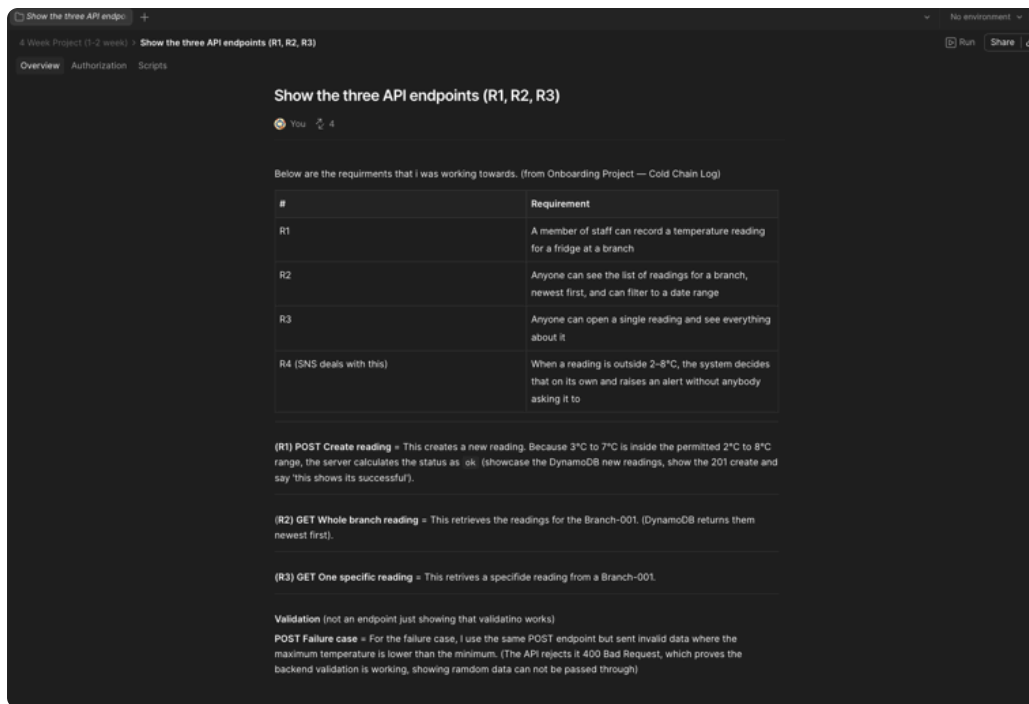
Research GSLI scan vs quires. =

Add API keys to making it more secure (change te HTTPS to RESTful APIs and chek setting to see i you can change it to private). =

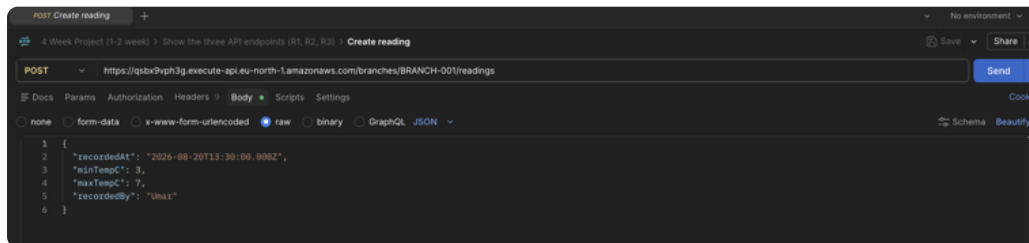
Postman screenshots

this is what I was showing the team, postman had the API request I was sending and infomration I was reading out to the team too. Shown bellow:

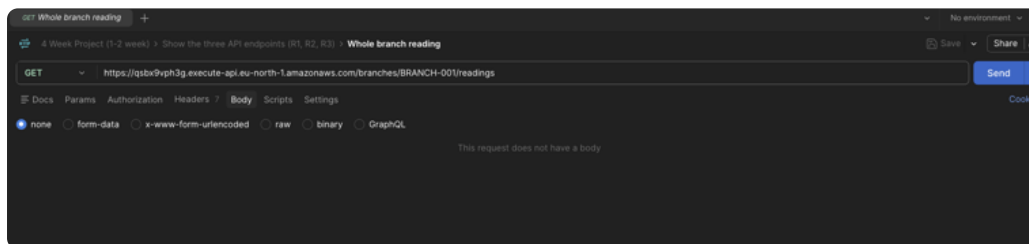




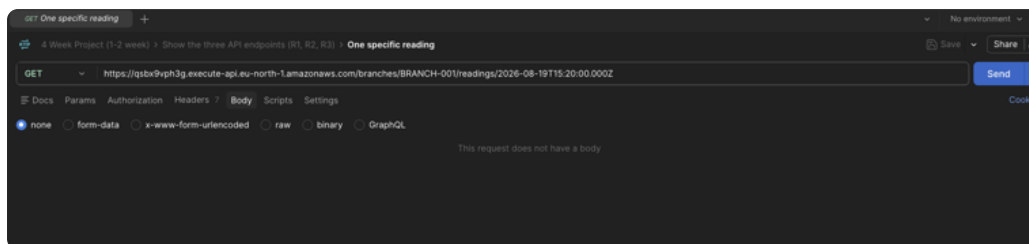
POST / create request:



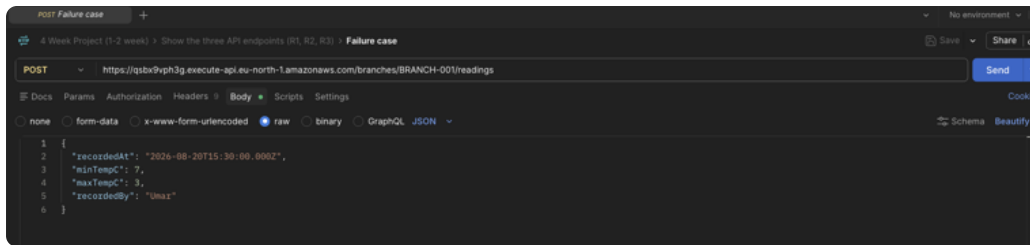
GET whole branch:



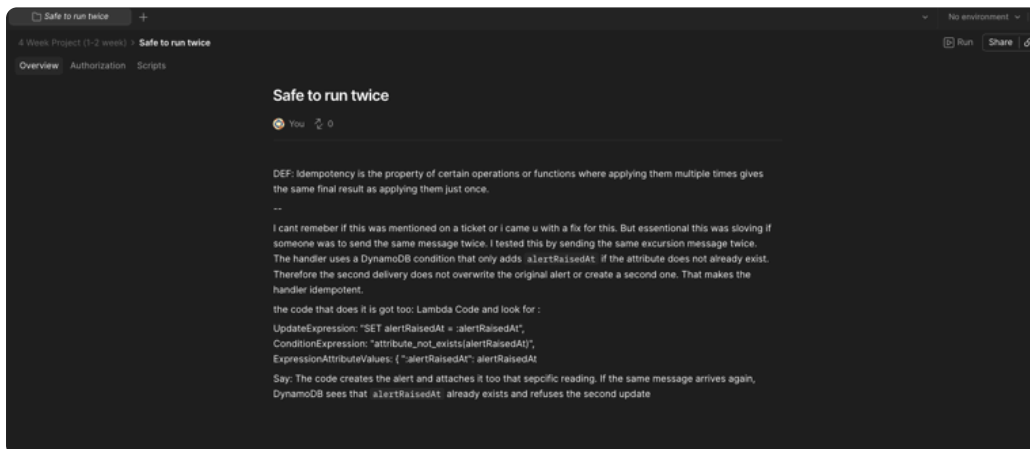
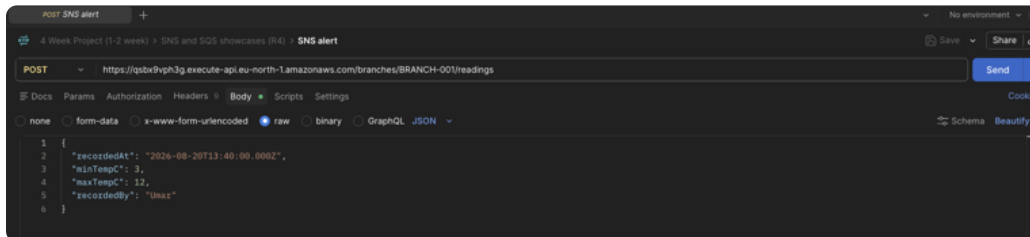
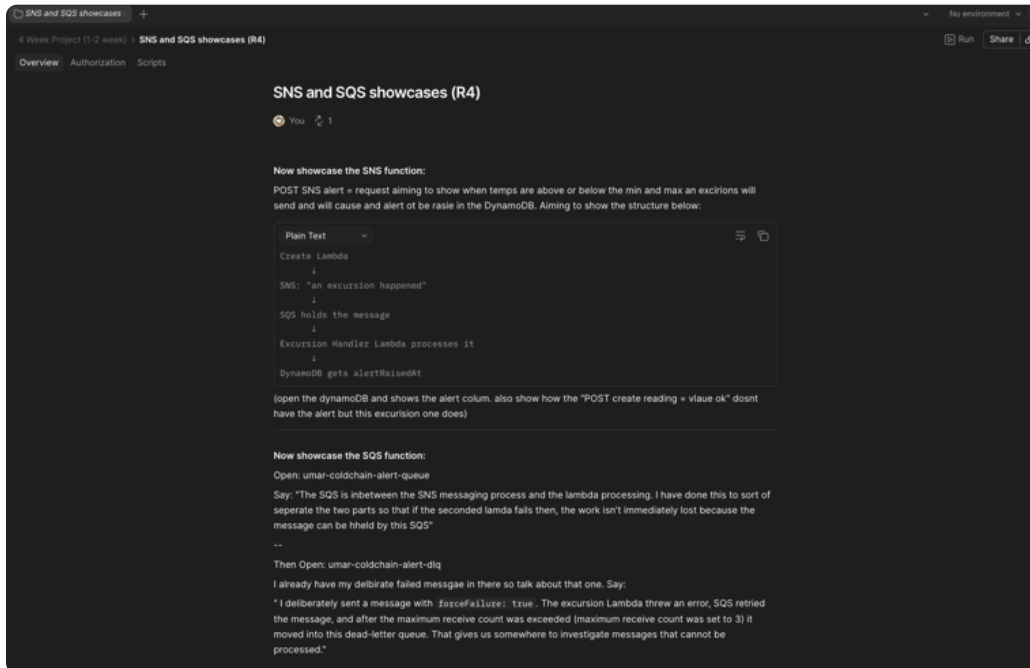
GET single reading:



Fail request (shows the project has data validation):



Demonstrates the SNS alert:



What I found difficult and

No environment

4 Week Project (1-2 week) > What I found difficult and what would I improve

OverviewAuthorizationScripts

What I found difficult and what would I improve

You0

I am also going to be completely honest and say that a lot of the more advanced JavaScript code in this project was written with the help of AI. In particular, things such as the asynchronous AWS SDK code, the SNS and SQS handling, and especially the DynamoDB conditional update used for idempotency were beyond what I would currently be able to write completely by myself. I am still learning how to write JavaScript properly, so I do not want to pretend that I wrote every line without help. However, I have tried to understand what the code is doing rather than just copying it. This project has massively increased my understanding of coding and AWS compared with where I was at the beginning. For example, when I first started the project I did not even properly understand basic things such as what a Lambda handler was, whereas now I can explain what the handler does, what the event contains, how the Lambda communicates with other AWS services and how the different parts of the system connect together.

Another area I found difficult was understanding the asynchronous messaging flow between SNS, SQS and the excursion-handler Lambda. I had to understand that SNS is responsible for publishing the excursion event, SQS stores the message until it can be processed, and then the second Lambda consumes that message and updates the DynamoDB record. This was different from the API Gateway flow because the alert process does not happen as one direct request from start to finish. Instead, each part of the system completes its own job and passes the work on to the next service.

CloudWatch also became very important while I was troubleshooting the project. At one point, the excursion-handler Lambda was receiving the SQS message correctly, but CloudWatch showed an `AccessDeniedException`. This helped me understand that the SNS and SQS parts were working and that the problem was specifically with the Lambda not having permission to update DynamoDB. After adding the required `dynamodb:updateItem` permission to the Lambda execution role, the alert flow worked and `alertRaisedAt` appeared automatically on the DynamoDB record.

I also found the idea of retries, visibility timeouts and dead-letter queues slightly difficult at first. I learned that when a Lambda fails to process an SQS message, the message is not immediately lost. It becomes available again after the visibility timeout and can be retried. If it continues to fail after the configured number of receives, it is moved into the dead-letter queue so that the failed message can be investigated instead of being retried forever.

Another concept I had to spend time understanding was idempotency. Because SQS uses at-least-once delivery, the same message can sometimes be delivered more than once. This means the Lambda has to be safe to run twice. I solved this by using a DynamoDB `ConditionExpression` that only sets `alertRaisedAt` if that field does not already exist. This means the first message raises the alert, but if the same message is delivered again, DynamoDB prevents the timestamp from being overwritten and the system does not create a duplicate alert.

MAIN MISTKE THAT WASNT SLOVED

MAIN MISTKE THAT WASNT SLOVED

You might have noticed through this demo, the requests don't have API keys yet, they still work. Luke pointed this out to me last Friday.

This is the one mistake that was made that I didn't fix was that API requests could be sent from Postman without the need for the API key from the gateway. The system itself is working correctly, but the API is currently publicly callable by anyone who knows the invoke URL. I understand this is a major security and authenticity risk. The functionality is complete and working, but authentication is one area I would revisit. At the moment the HTTP API is publicly callable if someone knows the URL. I investigated API keys, but adding AWS's native API-key functionality would mean changing the API Gateway type, so I chose not to risk breaking the working project just before the demo.

RAI Improvements = why it is taking so long to load. Something to do with the dql, what is five four message. GSI scan vs GSI query.